

Parking Ticket Simulator Project Report

Name: Emory Bruington

IDE used: Visual Studio

GitHub repository URL: <https://github.com/l4mont/csc223-parking-ticket-simulator>

Project Description

This project simulates a police officer checking whether a car has stayed too long at a parking meter. I divided the program into four classes. `ParkedCar` stores the vehicle information and how long it has been parked. `ParkingMeter` stores the amount of time that was purchased. `PoliceOfficer` compares those times. `ParkingTicket` stores the information needed for a citation, including the illegal minutes and fine.

The officer receives the car and meter through `inspect_car`. If the car is still within the purchased time, the method returns `None`. If the car is over the time limit, it calculates the illegal minutes and creates a `ParkingTicket`. The fine is \$25 for the first hour or part of an hour, plus \$10 for each additional hour or part of an hour.

The unit tests demonstrate valid behavior, rejected values, fine boundaries, ticket-report content, and collaboration between the objects.

Planning

Class Responsibility Plan

Class	Primary responsibility	Data stored	Important methods/properties
<code>ParkedCar</code>	Represent a car parked at a meter	Make, model, color, license number, minutes parked	Validated properties
<code>ParkingMeter</code>	Record purchased parking time	Minutes purchased	Validated property
<code>ParkingTicket</code>	Represent an issued citation	Copied car/officer data, illegal minutes	fine, <code>get_report</code>
<code>PoliceOfficer</code>	Inspect the car and decide whether to issue a ticket	Name and badge number	<code>inspect_car</code>

Collaboration Plan

`main.py` creates a `ParkedCar`, `ParkingMeter`, and `PoliceOfficer`. It passes the car and meter to `officer.inspect_car(car, meter)`. The officer compares the two minute values. If a violation exists, the officer creates a `ParkingTicket` and returns it; otherwise, the officer returns `None`. The ticket receives the existing car and officer as constructor parameters and copies the required values into the ticket. Later changes to those objects do not change the issued citation.

Unit Test Plan

The tests use Arrange-Act-Assert. They cover valid construction and property access, valid reassignment, invalid strings and numbers, no-ticket cases, ticket creation, all required fine boundaries, collaboration values, and report content.

Design and Implementation

ParkedCar, ParkingMeter, and PoliceOfficer use protected attributes with validated public properties. Their constructors reuse the setters. ParkingTicket validates positive illegal minutes and exposes its copied data through read-only properties. Incorrect value types raise TypeError, and invalid values raise ValueError. Boolean values are rejected as minutes even though Python treats bool as a subclass of int. The officer subtracts purchased minutes from parked minutes. The ticket calculates its fine as $25 + ((\text{illegal_minutes} - 1) // 60) * 10$. This charges \$25 for minutes 1 through 60, \$35 for 61 through 120, and \$45 for 121 through 180 without using floating-point rounding.

PoliceOfficer imports ParkedCar, ParkingMeter, and ParkingTicket. ParkingTicket reads the supplied objects without importing PoliceOfficer, so there is no circular import. Copying the data keeps the citation stable; the direction of the imports is what avoids a circular dependency.

The ticket uses a copying design rather than storing live references to the car and officer. This means later changes to the independently existing car or officer do not rewrite an already issued citation. Dependency injection is used because the car and meter are supplied to inspect_car instead of being created inside PoliceOfficer.

Unit Testing

Testing Environment

IDE: Visual Studio. The verification run used Python 3.12.14 and the built-in unittest module from the project terminal. No third-party packages are required. The tests folder contains an empty __init__.py and four test modules named test_*.py. Run `python -m unittest discover -s tests -v` from the project root. The final suite contains 26 test methods; subTest loops check additional inputs within those methods. All 26 tests passed. In Visual Studio, select a Python interpreter and set main.py as the startup file; the same terminal command runs the full suite.

Testing Strategy

The initial tests were added with the implementation. During final review, the suite was expanded to cover every car string property, invalid minute types, officer identity, and invalid inspection dependencies. For Arrange-Act-Assert, a test first creates a car parked for 61 minutes and a meter with 60 purchased minutes, then calls inspect_car, then checks that the result is a ParkingTicket with one illegal minute and a \$25 fine. assertEquals checks values, assertIsNone checks legal parking, assertIsInstance checks the ticket type, assertRaises checks validation, and assertIn checks report fields. The new dependency test failed before the inspection method was corrected and passed afterward.

Required Test Results

Test or feature	Input	Expected result	Actual result	Pass/Fail	Comment
Valid parked car	Toyota, Camry, 30 minutes	Correct properties	Correct properties	Pass	
Invalid parked minutes	-1 or 1.5	Exception	ValueError/TypeError	Pass	
Valid meter	60 purchased	60 minutes	60 minutes	Pass	Property access
Car under purchased time	59 parked, 60 purchased	None	None	Pass	No citation

Test or feature	Input	Expected result	Actual result	Pass/Fail	Comment
Car exactly at meter	60 parked, 60 purchased	None	None	Pass	
One illegal minute	61 parked, 60 purchased	\$25	\$25	Pass	
Sixty illegal minutes	60 illegal	\$25	\$25	Pass	Boundary
Sixty-one illegal minutes	121 parked, 60 purchased	\$35	\$35	Pass	
One hundred twenty illegal minutes	120 illegal	\$35	\$35	Pass	Boundary
One hundred twenty-one illegal minutes	181 parked, 60 purchased	\$45	\$45	Pass	
Ticket car information	Toyota Camry, Blue, ABC-1234	Matching data	Matching data	Pass	All four fields
Ticket officer information	Jordan Lee, A-1042	Matching data	Matching data	Pass	Name and badge
Ticket report	Issued ticket	All required fields	All required fields	Pass	

Problems and Corrections

During final review, `test_wrong_collaborator_types_raise_type_error` passed `None` as the car. The expected result was `TypeError`, following the `incorrect-type` rule. The actual result was `AttributeError` because `inspect_car` tried to read `minutes_parked` before checking the supplied object. The correction added `isinstance` checks for `ParkedCar` and `ParkingMeter` before reading their properties. The same test checks an invalid meter too. After the correction, the test and the complete 26-test suite passed.

GitHub correction commit: `f899c7f828fe13635a926cf85ca014c3beed14be`, titled `Fix invalid inspection dependencies to raise TypeError`. The expanded tests are also committed in the repository. This was a validation defect; the original five fine-boundary cases already passed.

Reflection

How did separating the program into four classes improve the design? Each class has one focused responsibility, so validation, fine calculation, and collaboration logic are easier to understand and test independently.

What complete collaboration occurs? The officer receives the car and meter, compares parked and purchased minutes, calculates illegal minutes when needed, creates a ticket, and returns that ticket or `None`.

Where was dependency injection used? The car and meter are passed to `inspect_car`. This keeps object creation outside the officer and makes tests able to supply controlled objects.

Was aggregation used? The final design copies the needed data instead of retaining references. The car and officer exist independently, and the ticket captures a stable citation snapshot.

Was composition used? No unnecessary composition was added. The project does not need an internally owned helper object.

Why create a ticket only after finding a violation? A ticket requires positive illegal minutes. Checking first avoids issuing a citation for legal parking or constructing an invalid ticket.

Why return a ticket or None? A returned object or None can be checked directly with unittest, unlike a method that only prints output.

Which boundary needed the most careful testing? The transition from 60 to 61 illegal minutes is important because 61 minutes is a partial second hour and must increase the fine.

What did the tests reveal? The collaboration tests verify that information actually travels from the independently created car and officer into the returned ticket.

How did unit tests affect development? They made the expected return values and exceptions explicit. Expanding the tests during review exposed missing type checks and confirmed that changing the car or officer afterward does not change an existing ticket.

Which test failed during development? The invalid-dependency test failed during final review because None caused AttributeError. Adding type checks changed the result to the expected TypeError. That showed why the method should validate its inputs before using their properties.

How did docstrings help? They describe responsibilities, inputs, outputs, validation rules, and collaboration behavior so each class can be used without reading every implementation detail.

What was learned? Python objects collaborate cleanly when each class has a narrow responsibility and dependencies are passed explicitly.